

MANUAL INTEGRAL DE CONSTRUCCIÓN, MODIFICACIÓN, MANTENIMIENTO Y EVOLUCIÓN DE OMNIMERGE

Guía técnica para reconstruir, modificar, ampliar, asegurar, transferir y mantener el proyecto Laravel

Proyecto: OmniMerge

Framework: Laravel 12

Lenguaje: PHP 8.2 o superior

Base de datos: MySQL

Frontend: Blade, Tailwind CSS, Alpine.js y Vite

Repositorio: <https://github.com/grverlearner/OmniMerge.git>

Rama principal: [main](#)

Fecha de actualización: agosto de 2026

1. PROPÓSITO DE ESTE MANUAL

Este documento explica cómo está construido OmniMerge y qué debe modificarse según el tipo de cambio que se quiera realizar.

El manual sirve para:

- Reconstruir el proyecto desde cero.
- Instalar el proyecto en otra computadora.
- Comprender su arquitectura.
- Identificar dónde se encuentra cada módulo.
- Crear nuevos módulos.
- Añadir campos a una tabla.
- Modificar formularios.
- Cambiar vistas.
- Cambiar validaciones.
- Cambiar permisos.
- Crear nuevas relaciones.
- Subir imágenes.
- Añadir rutas.
- Eliminar funcionalidades.
- Transferir el proyecto.
- Actualizar dependencias.
- Corregir errores.
- Proteger la seguridad.
- Crear pruebas.
- Preparar nuevas funcionalidades.

Este documento no sustituye al código. Su función es explicar el esqueleto del proyecto, la responsabilidad de cada carpeta y el procedimiento correcto para realizar cambios.

2. REGLA PRINCIPAL DE OMNIMERGE

En OmniMerge, una funcionalidad normalmente no se encuentra dentro de un solo archivo. Por ejemplo, para añadir un nuevo campo llamado **featured** a las colecciones, probablemente sea necesario modificar:

Base de datos
↓
Modelo
↓
Request
↓
Controlador
↓
Formulario
↓
Vista de detalle
↓
Pruebas

No se debe modificar únicamente la vista y esperar que el dato se guarde.
Una funcionalidad completa puede involucrar las siguientes capas:

Capa	Responsabilidad
Migración	Cambiar la estructura de la base de datos
Modelo	Representar datos y relaciones
Form Request	Validar la entrada
Policy	Autorizar al usuario
Controller	Coordinar la acción
Service	Ejecutar lógica compleja
Route	Exponer una URL
Blade	Mostrar la interfaz
Test	Comprobar el funcionamiento

Laravel permite agrupar las operaciones CRUD en controladores de recurso y registrarlas mediante `Route::resource()`. También ofrece Form Requests para encapsular validación y autorización, Policies para organizar permisos y migraciones para versionar la estructura de la base de datos.

3. ESTADO ACTUAL DEL PROYECTO

OmniMerge ya cuenta o ha sido preparado para contar con los siguientes módulos:

Autenticación
Dashboard
Perfil
Tipos de entidad

Entidades
Atributos dinámicos
Opciones de atributos
Grupos de atributos
Valores de entidades
Colecciones
Exploración comunitaria
Clonación de contenido
Carga de imágenes
Visibilidad pública y privada
Policies de seguridad

También está preparado conceptualmente para:

Universos
Temporadas
Torneos
Simulaciones
Rankings
Relaciones entre atributos
Condiciones
Favoritos
Comentarios
Moderación
Notificaciones

4. ESQUELETO PRINCIPAL DEL PROYECTO

La estructura general es:

OmniMerge/
|— app/
|— bootstrap/
|— config/
|— database/
|— public/
|— resources/
|— routes/
|— storage/
|— tests/
|— vendor/
|— node_modules/
|— .env
|— .env.example
|— artisan
|— composer.json

```
|— composer.lock
|— package.json
|— package-lock.json
|— phpunit.xml
|— tailwind.config.js
|— vite.config.js
```

4.1. Carpetas que se modifican frecuentemente

```
app/
database/
resources/
routes/
tests/
config/
```

4.2. Carpetas que normalmente no deben modificarse manualmente

```
vendor/
node_modules/
bootstrap/cache/
storage/framework/
public/build/
```

Estas carpetas contienen dependencias, archivos generados o caché.

4.3. Archivos que no deben publicarse

.env

El archivo **.env** puede contener:

- Contraseña de MySQL.
- Claves privadas.
- Tokens.
- Credenciales.
- Configuración local.

Nunca debe subirse a GitHub.

4.4. Archivos que sí deben publicarse

```
.env.example
composer.json
composer.lock
package.json
package-lock.json
database/migrations/
app/
resources/
routes/
tests/
```

5. FLUJO INTERNO DE UNA SOLICITUD

Cuando un usuario abre una página o envía un formulario, Laravel procesa la solicitud de esta forma:

Navegador



Ruta



Middleware



Controlador



Policy o Form Request



Modelo o Service



Base de datos



Vista Blade



Respuesta al navegador

Ejemplo:

Usuario abre /collections



routes/web.php



CollectionController@index



Collection::query()



Base de datos MySQL



collections/index.blade.php

Otro ejemplo:

Usuario guarda una colección



POST /collections



StoreCollectionRequest



CollectionPolicy

↓
CollectionController@store
↓
Collection::create()
↓
collection_entity
↓
Redirección

6. RECONSTRUIR OMNIMERGE DESDE CERO

Esta sección explica cómo crear nuevamente el proyecto desde una instalación limpia.

6.1. Requisitos

Instalar:

PHP 8.2 o superior

Composer

Node.js

NPM

MySQL

Git

Visual Studio Code

Comprobar:

`php -v`

`composer --version`

`node -v`

`npm -v`

`git --version`

6.2. Crear proyecto Laravel

`composer create-project laravel/laravel OmniMerge`

`cd OmniMerge`

Para trabajar específicamente con Laravel 12:

`composer create-project laravel/laravel OmniMerge "^12.0"`

6.3. Instalar Breeze

`composer require laravel/breeze --dev`

`php artisan breeze:install blade`

Después:

`npm.cmd install`

`npm.cmd run build`

6.4. Configurar .env

Copiar:

Copy-Item .env.example .env

Generar clave:

php artisan key:generate

Configurar:

APP_NAME=OmniMerge

APP_ENV=local

APP_DEBUG=true

APP_URL=http://127.0.0.1:8000

APP_LOCALE=es

APP_FALLBACK_LOCALE=es

APP_FAKER_LOCALE=es_ES

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=omnimerge

DB_USERNAME=root

DB_PASSWORD=

SESSION_DRIVER=database

CACHE_STORE=database

QUEUE_CONNECTION=database

FILESYSTEM_DISK=local

6.5. Crear base de datos

CREATE DATABASE omnimerge

CHARACTER SET utf8mb4

COLLATE utf8mb4_unicode_ci;

6.6. Ejecutar migraciones

php artisan migrate

6.7. Crear enlace de imágenes

php artisan storage:link

6.8. Ejecutar proyecto

Terminal 1:

php artisan serve

Terminal 2:
npm.cmd run dev

Abrir:
<http://127.0.0.1:8000>

7. INSTALAR EL PROYECTO DESDE GITHUB

Para instalar el proyecto existente:
git clone <https://github.com/grverlearner/OmniMerge.git>
cd OmniMerge

Instalar dependencias:
composer install
npm.cmd install

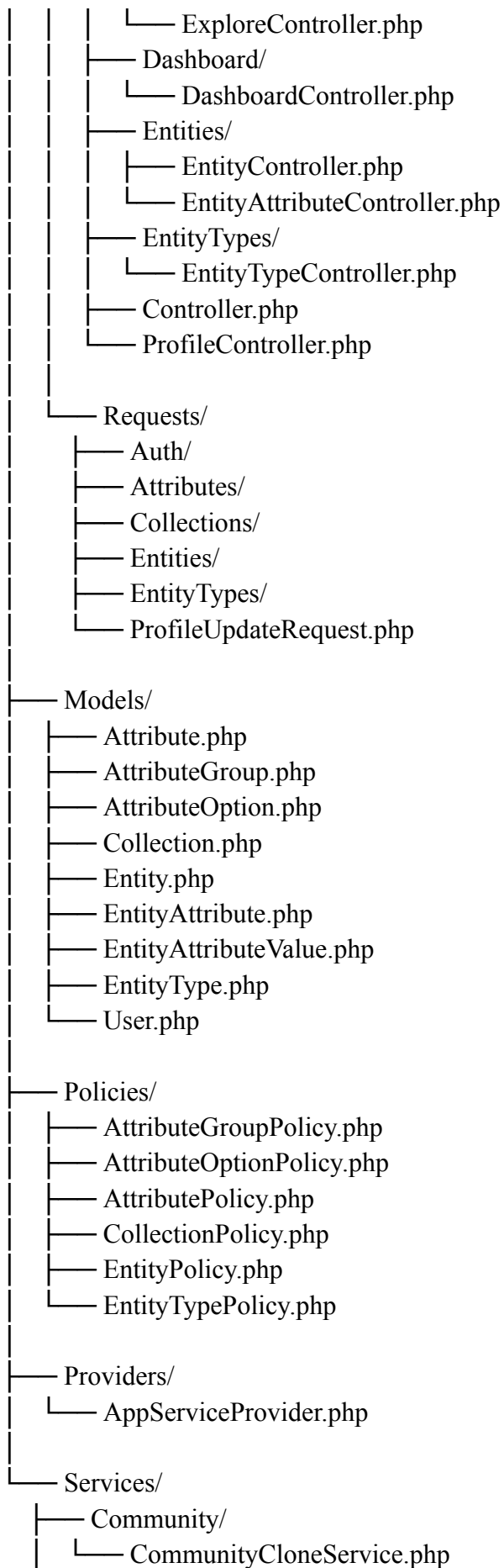
Crear entorno:
Copy-Item .env.example .env
php artisan key:generate

Configurar MySQL y luego ejecutar:
php artisan migrate
php artisan storage:link
npm.cmd run build
php artisan optimize:clear

Ejecutar:
php artisan serve
npm.cmd run dev

8. ESTRUCTURA ACTUAL DE **app**

```
app/
├── Http/
│   ├── Controllers/
│   │   ├── Auth/
│   │   ├── Attributes/
│   │   │   ├── AttributeController.php
│   │   │   ├── AttributeGroupController.php
│   │   │   └── AttributeOptionController.php
│   │   ├── Collections/
│   │   │   └── CollectionController.php
│   │   └── Community/
```

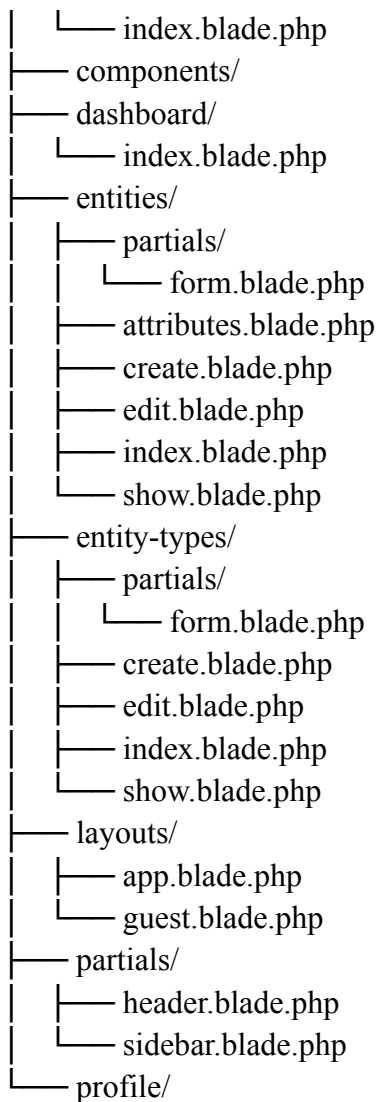



- └─ Entities/
 - └─ EntityAttributeValueService.php

9. ESTRUCTURA ACTUAL DE VISTAS

resources/views/

- └─ auth/
- └─ attribute-groups/
 - └─ partials/
 - └─ form.blade.php
 - └─ create.blade.php
 - └─ edit.blade.php
 - └─ index.blade.php
 - └─ show.blade.php
- └─ attribute-options/
 - └─ partials/
 - └─ form.blade.php
 - └─ create.blade.php
 - └─ edit.blade.php
 - └─ index.blade.php
 - └─ show.blade.php
- └─ attributes/
 - └─ partials/
 - └─ form.blade.php
 - └─ create.blade.php
 - └─ edit.blade.php
 - └─ index.blade.php
 - └─ show.blade.php
- └─ collections/
 - └─ partials/
 - └─ form.blade.php
 - └─ create.blade.php
 - └─ edit.blade.php
 - └─ index.blade.php
 - └─ show.blade.php
- └─ community/
 - └─ partials/
 - └─ attribute-card.blade.php
 - └─ collection-card.blade.php
 - └─ entity-card.blade.php
 - └─ filters.blade.php
 - └─ attribute.blade.php
 - └─ collection.blade.php
 - └─ entity.blade.php



10. QUÉ ARCHIVO MODIFICAR SEGÚN EL CAMBIO

Esta tabla permite identificar rápidamente qué componente tocar.

Cambio solicitado	Archivos principales
Cambiar un texto	Vista Blade
Cambiar colores o diseño	Vista Blade y Tailwind
Añadir enlace al menú	partials/sidebar.blade.php
Añadir página	Ruta, controlador y vista
Añadir campo	Migración, modelo, Request, controlador, vista
Cambiar validación	Form Request
Cambiar permisos	Policy

Cambiar consulta	Controlador o scope del modelo
Cambiar relación	Migración y modelos
Añadir carga de archivo	Migración, Request, controlador, modelo y formulario
Añadir lógica compleja	Service
Añadir búsqueda	Controlador y vista
Añadir filtro	Controlador, vista e índice de base de datos
Añadir contenido público	Modelo, Request, controlador, Policy y vista
Añadir clonación	Service, controlador, rutas y pruebas
Añadir contador	Migración, controlador o evento
Añadir nuevo módulo CRUD	Todos los componentes
Borrar módulo	Rutas, vistas, controlador, modelo, requests, policy y migración nueva
Cambiar base de datos	Nueva migración
Corregir seguridad	Policy, Request, consultas y pruebas

11. CAMBIOS PEQUEÑOS, MEDIANOS Y GRANDES

11.1. Cambio pequeño

Ejemplo:

Cambiar el texto “Nueva colección” por “Crear colección”

Modificar solamente:

`resources/views/collections/index.blade.php`

No es necesaria una migración.

11.2. Cambio mediano

Ejemplo:

Añadir un campo destacado a las colecciones

Modificar:

Migración

Modelo

Request

Formulario

Controlador

Vista

Test

11.3. Cambio grande

Ejemplo:

Crear módulo de universos

Crear:

Migración

Modelo

Factory

Controlador

Requests

Policy

Service, si es necesario

Rutas

Vistas

Tests

Sidebar

Dashboard

12. CÓMO CREAR UN MÓDULO NUEVO

Se utilizará el futuro módulo **Universe** como ejemplo.

12.1. Generar modelo, migración y factory

```
php artisan make:model Universe -mf
```

Esto crea:

app/Models/Universe.php

database/migrations/..._create_universes_table.php

database/factories/UniverseFactory.php

12.2. Crear controlador CRUD

```
php artisan make:controller Universes/UniverseController --resource --model=Universe
```

12.3. Crear Requests

```
php artisan make:request Universes/StoreUniverseRequest
```

```
php artisan make:request Universes/UpdateUniverseRequest
```

12.4. Crear Policy

```
php artisan make:policy UniversePolicy --model=Universe
```

12.5. Crear vistas

```
New-Item -ItemType Directory -Force resources\views\universes
```

```
New-Item -ItemType Directory -Force resources\views\universes\partials
```

```
New-Item -ItemType File resources\views\universes\index.blade.php
```

```
New-Item -ItemType File resources\views\universes\create.blade.php
```

```
New-Item -ItemType File resources\views\universes\edit.blade.php
```

```
New-Item -ItemType File resources\views\universes\show.blade.php
```

```
New-Item -ItemType File resources\views\universes\partials\form.blade.php
```

12.6. Crear migración

```
Schema::create('universes', function (Blueprint $table) {
```

```
    $table->id();
```

```
    $table->foreignId('user_id')
```

```
        ->constrained()
```

```
        ->cascadeOnDelete();
```

```
    $table->string('name', 150);
```

```
    $table->string('code', 50);
```

```
    $table->string('slug', 180);
```

```
    $table->text('description')->nullable();
```

```
    $table->string('image')->nullable();
```

```
    $table->string('visibility', 20)
```

```
        ->default('PRIVATE');
```

```
    $table->string('status', 20)
```

```
        ->default('ACTIVE');
```

```
    $table->json('configuration')->nullable();
```

```
    $table->timestamps();
```

```
    $table->softDeletes();
```

```
    $table->unique(['user_id', 'code']);
```

```
    $table->unique(['user_id', 'slug']);
```

```
});
```

12.7. Modelo

```
<?php
```

```
namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Factories\HasFactory;
```

```

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\SoftDeletes;

```

```

class Universe extends Model
{
    use HasFactory;
    use SoftDeletes;

    protected $fillable = [
        'user_id',
        'name',
        'code',
        'slug',
        'description',
        'image',
        'visibility',
        'status',
        'configuration',
    ];

    protected function casts(): array
    {
        return [
            'configuration' => 'array',
        ];
    }

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}

```

12.8. Relación en **User.php**

```

use Illuminate\Database\Eloquent\Relations\HasMany;

```

```

public function universes(): HasMany
{
    return $this->hasMany(Universe::class);
}

```

12.9. Request

```

<?php

```

```

namespace App\Http\Requests\Universes;

use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Support\Str;
use Illuminate\Validation\Rule;

class StoreUniverseRequest extends FormRequest
{
    public function authorize(): bool
    {
        return $this->user()?->isActive() === true;
    }

    protected function prepareForValidation(): void
    {
        $name = trim((string) $this->input('name'));

        $this->merge([
            'name' => $name,
            'code' => Str::upper(
                Str::slug(
                    $this->input('code') ?: $name,
                    '-'
                )
            ),
            'slug' => Str::slug(
                $this->input('slug') ?: $name
            ),
        ]);
    }

    public function rules(): array
    {
        return [
            'name' => [
                'required',
                'string',
                'max:150',
            ],

            'code' => [
                'required',
                'string',
            ],
        ];
    }
}

```



```

        'max:50',

        Rule::unique('universes', 'code')
            ->where(
                fn ($query) => $query->where(
                    'user_id',
                    $this->user()->id
                )
            ),
    ],

    'slug' => [
        'required',
        'string',
        'max:180',
    ],

    'description' => [
        'nullable',
        'string',
        'max:5000',
    ],

    'visibility' => [
        'required',
        Rule::in([
            'PRIVATE',
            'PUBLIC',
            'UNLISTED',
        ]),
    ],

    'status' => [
        'required',
        Rule::in([
            'ACTIVE',
            'INACTIVE',
            'ARCHIVED',
        ]),
    ],
];
}
}

```

Los Form Requests deben utilizarse cuando la validación empieza a crecer o necesita reglas de autorización propias. Laravel los coloca en `app/Http/Requests` y permite recuperar únicamente datos validados mediante `$request->validated()`.

12.10. Policy

```
<?php
```

```
namespace App\Policies;
```

```
use App\Models\Universe;
```

```
use App\Models\User;
```

```
class UniversePolicy
```

```
{
```

```
    public function viewAny(User $user): bool
```

```
    {
```

```
        return $user->isActive();
```

```
    }
```

```
    public function view(
```

```
        User $user,
```

```
        Universe $universe
```

```
    ): bool {
```

```
        return $universe->user_id === $user->id;
```

```
    }
```

```
    public function create(User $user): bool
```

```
    {
```

```
        return $user->isActive();
```

```
    }
```

```
    public function update(
```

```
        User $user,
```

```
        Universe $universe
```

```
    ): bool {
```

```
        return $universe->user_id === $user->id;
```

```
    }
```

```
    public function delete(
```

```
        User $user,
```

```
        Universe $universe
```

```
    ): bool {
```

```
        return $universe->user_id === $user->id;
```

```
    }
```

```
}
```

Las Políticas son apropiadas para organizar permisos relacionados con un modelo concreto, por ejemplo determinar si un usuario puede actualizar o eliminar un universo.

12.11. Controlador

```
<?php
```

```
namespace App\Http\Controllers\Universes;
```

```
use App\Http\Controllers\Controller;
use App\Http\Requests\Universes\StoreUniverseRequest;
use App\Models\Universe;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\View\View;
```

```
class UniverseController extends Controller
{
    public function index(Request $request): View
    {
        $this->authorize('viewAny', Universe::class);

        $universes = Universe::query()
            ->where('user_id', $request->user()->id)
            ->latest()
            ->paginate(12);

        return view(
            'universes.index',
            compact('universes')
        );
    }

    public function create(): View
    {
        $this->authorize('create', Universe::class);

        return view('universes.create');
    }

    public function store(
        StoreUniverseRequest $request
    ): RedirectResponse {
        $universe = $request
            ->user()
```

```

        ->universes()
        ->create($request->validated());

    return redirect()
        ->route('universes.show', $universe)
        ->with(
            'success',
            'Universo creado correctamente.'
        );
    }

    public function show(Universe $universe): View
    {
        $this->authorize('view', $universe);

        return view(
            'universes.show',
            compact('universe')
        );
    }
}

```

12.12. Rutas

En:

routes/web.php

Importar:

```
use App\Http\Controllers\Universes\UniverseController;
```

Agregar dentro de **auth**:

```
Route::resource(
    'universes',
    UniverseController::class
);
```

Una ruta de recurso genera las acciones convencionales **index**, **create**, **store**, **show**, **edit**, **update** y **destroy**.

12.13. Ejecutar

```
php artisan migrate
```

```
php artisan optimize:clear
```

```
php artisan route:list --name=universes
```

13. CÓMO AÑADIR UN CAMPO A UNA TABLA

Nunca se debe modificar directamente una migración antigua si ya fue ejecutada en otros entornos.

Se debe crear una migración nueva.

Ejemplo: añadir **featured** a **collections**.

13.1. Crear migración

```
php artisan make:migration add_featured_to_collections_table --table=collections
```

13.2. Código

```
public function up(): void
```

```
{  
    Schema::table('collections', function (Blueprint $table) {  
        $table->boolean('featured')  
            ->default(false)  
            ->after('status');  
    });  
}
```

```
public function down(): void
```

```
{  
    Schema::table('collections', function (Blueprint $table) {  
        $table->dropColumn('featured');  
    });  
}
```

Las migraciones actúan como control de versiones para la estructura de la base de datos.

Cada cambio debe poder aplicarse con **up()** y revertirse con **down()**.

13.3. Modelo

En:

app/Models/Collection.php

Añadir:

'featured',

En **casts()**:

'featured' => 'boolean',

13.4. Request

Añadir:

```
'featured' => [  
    'boolean',  
],
```

En **prepareForValidation()**:

'featured' => \$this->boolean('featured'),

13.5. Formulario

```
<label class="flex items-center gap-3">
  <input
    type="checkbox"
    name="featured"
    value="1"
    @checked(
      old(
        'featured',
        $collection->featured ?? false
      )
    )
  >

  <span>Destacar colección</span>
</label>
```

13.6. Vista

```
@if ($collection->featured)
  <span class="rounded-full bg-amber-100 px-3 py-1 text-xs font-bold text-amber-700">
    Destacada
  </span>
@endif
```

13.7. Ejecutar

```
php artisan migrate
php artisan optimize:clear
php artisan test
```

14. CÓMO CAMBIAR EL NOMBRE DE UN CAMPO

Ejemplo:

Cambiar description por summary

Crear migración:

```
php artisan make:migration rename_description_to_summary_in_collections_table
--table=collections
```

Código:

```
public function up(): void
{
    Schema::table('collections', function (Blueprint $table) {
        $table->renameColumn(
```

```

        'description',
        'summary'
    );
});
}

public function down(): void
{
    Schema::table('collections', function (Blueprint $table) {
        $table->renameColumn(
            'summary',
            'description'
        );
    });
}

```

Después modificar:

Modelo
 Requests
 Controlador
 Vistas
 Tests
 Factories
 Seeders
 Services

Buscar en todo el proyecto:

Ctrl + Shift + F

Buscar:

description

No debe cambiarse únicamente la columna de MySQL.

15. CÓMO ELIMINAR UN CAMPO

Ejemplo: eliminar **numeric_value** de opciones.

15.1. Revisar dependencias

Antes de eliminar, buscar:

numeric_value

Comprobar:

Modelo
 Request
 Formulario

Vista
Service
Tests
Seeders

15.2. Crear migración

```
php artisan make:migration drop_numeric_value_from_attribute_options_table  
--table=attribute_options
```

```
public function up(): void  
{  
    Schema::table('attribute_options', function (Blueprint $table) {  
        $table->dropColumn('numeric_value');  
    });  
}
```

```
public function down(): void  
{  
    Schema::table('attribute_options', function (Blueprint $table) {  
        $table->decimal(  
            'numeric_value',  
            15,  
            4  
        )->nullable();  
    });  
}
```

15.3. Eliminar referencias

Quitar de:

\$fillable
casts()
rules()
formularios
vistas
services
tests

15.4. Ejecutar primero en desarrollo

```
php artisan migrate --pretend  
php artisan migrate
```

migrate --pretend permite revisar las sentencias antes de ejecutarlas. Los comandos de rollback, refresh y fresh deben utilizarse con precaución porque pueden revertir o eliminar datos.

16. CÓMO AÑADIR UNA RELACIÓN

Ejemplo: una entidad puede tener comentarios.

16.1. Crear modelo y migración

```
php artisan make:model Comment -mf
```

16.2. Migración

```
Schema::create('comments', function (Blueprint $table) {  
    $table->id();  
  
    $table->foreignId('user_id')  
        ->constrained()  
        ->cascadeOnDelete();  
  
    $table->foreignId('entity_id')  
        ->constrained()  
        ->cascadeOnDelete();  
  
    $table->text('content');  
  
    $table->timestamps();  
    $table->softDeletes();  
});
```

16.3. Modelo **Comment**

```
public function entity(): BelongsTo  
{  
    return $this->belongsTo(Entity::class);  
}
```

```
public function user(): BelongsTo  
{  
    return $this->belongsTo(User::class);  
}
```

16.4. Modelo **Entity**

```
public function comments(): HasMany  
{  
    return $this->hasMany(Comment::class);  
}
```

16.5. Modelo **User**

```
public function comments(): HasMany  
{
```

```
    return $this->hasMany(Comment::class);
}
```

17. RELACIÓN MUCHOS A MUCHOS

Ejemplo actual:

Colecciones ↔ Entidades

Tabla:

collection_entity

Modelo **Collection**:

```
public function entities(): BelongsToMany
{
    return $this->belongsToMany(
        Entity::class,
        'collection_entity'
    )
    ->withPivot([
        'sort_order',
        'notes',
        'added_at',
    ])
    ->orderByPivot('sort_order');
}
```

Modelo **Entity**:

```
public function collections(): BelongsToMany
{
    return $this->belongsToMany(
        Collection::class,
        'collection_entity'
    )
    ->withPivot([
        'sort_order',
        'notes',
        'added_at',
    ])
    ->orderByPivot('sort_order');
}
```

Guardar:

```
$collection->entities()->sync([
    5 => [
        'sort_order' => 0,
```

```

        'added_at' => now(),
    ],
    8 => [
        'sort_order' => 1,
        'added_at' => now(),
    ],
]);

```

Añadir sin borrar los existentes:

```

$collection->entities()->syncWithoutDetaching([
    $entityId => [
        'sort_order' => 5,
        'added_at' => now(),
    ],
]);

```

Quitar:

```

$collection->entities()->detach($entityId);

```

18. CÓMO MODIFICAR UN MODELO

Los modelos se encuentran en:

app/Models/

18.1. **\$fillable**

Todo dato enviado mediante **create()** o **update()** debe estar permitido.

```

protected $fillable = [
    'user_id',
    'name',
    'description',
];

```

Si se añade una columna pero no se añade a **\$fillable**, Laravel puede ignorarla o lanzar una excepción de asignación masiva.

18.2. **casts()**

Convierte datos.

```

protected function casts(): array
{
    return [
        'allow_cloning' => 'boolean',
        'published_at' => 'datetime',
        'metadata' => 'array',
        'views_count' => 'integer',
    ];
}

```

```
}
```

18.3. Relaciones

```
public function user(): BelongsTo
{
    return $this->belongsTo(User::class);
}
```

18.4. Scopes

```
public function scopeOwnedBy(
    Builder $query,
    User $user
): Builder {
    return $query->where(
        'user_id',
        $user->id
    );
}
```

Uso:

```
Collection::query()
    ->ownedBy($request->user())
    ->get();
```

18.5. Accessors

```
public function getImageUrlAttribute(): ?string
{
    if (! $this->image) {
        return null;
    }

    /** @var \Illuminate\Filesystem\FilesystemAdapter $disk */
    $disk = Storage::disk('public');

    if (! $disk->exists($this->image)) {
        return null;
    }

    return $disk->url($this->image);
}
```

Uso:

```
{{ $collection->image_url }}
```

18.6. Métodos de dominio

```
public function canBeCloned(): bool
{
    return $this->visibility === 'PUBLIC'
        && $this->status === 'ACTIVE'
        && $this->published_at !== null
        && $this->allow_cloning;
}
```

Estos métodos expresan reglas del negocio y evitan repetir condiciones.

19. CONFLICTO CON LOS NOMBRES **Attribute** Y **Collection**

OmniMerge utiliza:

App\Models\Attribute

App\Models\Collection

Estos nombres pueden confundirse con:

\Attribute

Illuminate\Support\Collection

La solución es importar siempre el modelo correcto.

```
use App\Models\Attribute;
```

```
use App\Models\Collection;
```

Cuando sea necesario usar la colección de Laravel:

```
use Illuminate\Support\Collection as SupportCollection;
```

Ejemplo:

```
public function options(): SupportCollection
{
    return collect([]);
}
```

No se deben importar dos clases con el mismo nombre sin usar alias.

20. CÓMO MODIFICAR VALIDACIONES

Las validaciones se encuentran en:

app/Http/Requests/

20.1. Añadir campo obligatorio

```
'name' => [
    'required',
    'string',
    'max:150',
```

```
],
```

20.2. Campo opcional

```
'description' => [  
  'nullable',  
  'string',  
  'max:5000',  
],
```

20.3. Estado permitido

```
'status' => [  
  'required',  
  Rule::in([  
    'ACTIVE',  
    'INACTIVE',  
    'ARCHIVED',  
  ]),  
],
```

20.4. Validar propiedad

```
'entity_ids.*' => [  
  Rule::exists('entities', 'id')  
    ->where(  
      fn ($query) => $query  
        ->where(  
          'user_id',  
          $this->user()->id  
        )  
    ->whereNull('deleted_at')  
  ),  
],
```

Esto evita que el usuario envíe el ID de una entidad ajena.

20.5. Normalizar antes de validar

```
protected function prepareForValidation(): void  
{  
  $name = trim(  
    (string) $this->input('name')  
  );  
  
  $this->merge([  
    'name' => $name,  
  
    'code' => Str::upper(  

```

```

        Str::slug(
            $this->input('code') ?: $name,
            '-'
        )
    ),

    'slug' => Str::slug(
        $this->input('slug') ?: $name
    ),

    'allow_cloning' =>
        $this->boolean('allow_cloning'),
]);
}

```

20.6. Mensajes

```

public function messages(): array
{
    return [
        'name.required' =>
            'El nombre es obligatorio.',

        'image.max' =>
            'La imagen no puede superar los 4 MB.',
    ];
}

```

21. CÓMO MODIFICAR PERMISOS

Las Políticas se encuentran en:

app/Policies/

21.1. Solo propietario

```

public function update(
    User $user,
    Collection $collection
): bool {
    return $collection->user_id === $user->id;
}

```

21.2. Propietario o público

```

public function view(
    User $user,
    Collection $collection

```

```

): bool {
    return $collection->user_id === $user->id
        || $collection->visibility === 'PUBLIC';
}

```

21.3. Solo administrador

```

public function delete(
    User $user,
    Collection $collection
): bool {
    return $user->role === 'ADMIN';
}

```

21.4. Propietario o administrador

```

public function delete(
    User $user,
    Collection $collection
): bool {
    return $collection->user_id === $user->id
        || $user->role === 'ADMIN';
}

```

21.5. Uso en controlador

```

$this->authorize(
    'update',
    $collection
);

```

21.6. Uso en Blade

```

@can('update', $collection)
    <a href="{{ route(
        'collections.edit',
        $collection
    ) }}">
        Editar
    </a>
@endcan

```

La interfaz no sustituye la seguridad. Aunque el botón se oculte en Blade, el controlador debe volver a comprobar la Policy.

22. CONTROLADOR BASE

En Laravel 12, el controlador base puede estar vacío. OmniMerge necesita:


```
app/Http/Controllers/Controller.php
```

```
<?php
```

```
namespace App\Http\Controllers;
```

```
use Illuminate\Foundation\Auth\Access\AuthorizesRequests;
```

```
abstract class Controller
{
    use AuthorizesRequests;
}
```

Esto habilita:

```
$this->authorize(...)
```

23. CÓMO MODIFICAR UN CONTROLADOR

Los controladores se encuentran en:

```
app/Http/Controllers/
```

23.1. **index**

Lista y filtra.

```
public function index(Request $request): View
```

```
{
    $search = trim(
        (string) $request->input('search')
    );

    $collections = Collection::query()
        ->ownedBy($request->user())
        ->when(
            $search,
            fn ($query) => $query->where(
                'name',
                'like',
                "%{$search}%"
            )
        )
        ->latest()
        ->paginate(12)
        ->withQueryString();
```

```
return view(
```

```

        'collections.index',
        compact(
            'collections',
            'search'
        )
    );
}

```

23.2. create

Carga datos para el formulario.

```

public function create(Request $request): View
{
    $entities = Entity::query()
        ->ownedBy($request->user())
        ->orderBy('name')
        ->get();

    return view(
        'collections.create',
        compact('entities')
    );
}

```

23.3. store

Guarda.

```

public function store(
    StoreCollectionRequest $request
): RedirectResponse {
    $data = $request->validated();

    $collection = $request
        ->user()
        ->collections()
        ->create($data);

    return redirect()
        ->route(
            'collections.show',
            $collection
        )
        ->with(
            'success',
            'Colección creada correctamente.'
        );
}

```

```
}
```

23.4. show

Muestra.

```
public function show(
    Collection $collection
): View {
    $this->authorize(
        'view',
        $collection
    );

    $collection->load([
        'entities.entityType',
    ]);

    return view(
        'collections.show',
        compact('collection')
    );
}
```

23.5. update

Actualiza.

```
public function update(
    UpdateCollectionRequest $request,
    Collection $collection
): RedirectResponse {
    $collection->update(
        $request->validated()
    );

    return redirect()
        ->route(
            'collections.show',
            $collection
        )
        ->with(
            'success',
            'Colección actualizada.'
        );
}
```

23.6. destroy

Elimina.

```
public function destroy(
    Collection $collection
): RedirectResponse {
    $this->authorize(
        'delete',
        $collection
    );

    $collection->delete();

    return redirect()
        ->route('collections.index')
        ->with(
            'success',
            'Colección eliminada.'
        );
}
```

24. CUÁNDO CREAR UN SERVICE

No toda lógica debe permanecer en el controlador.

Crear un Service cuando exista:

- Clonación de varios registros.
- Guardado de atributos tipados.
- Procesamiento de imágenes.
- Cálculos complejos.
- Simulación.
- Importación.
- Exportación.
- Operaciones con varias tablas.
- Transacciones extensas.

Ejemplos actuales:

app/Services/Community/CommunityCloneService.php

app/Services/Entities/EntityAttributeValueService.php

24.1. Ejemplo

```
class PublishCollectionService
{
    public function publish(
        Collection $collection
    ): Collection {
        return DB::transaction(
            function () use ($collection) {
```

```

        $collection->update([
            'visibility' => 'PUBLIC',
            'published_at' => now(),
        ]);

        return $collection;
    }
);
}
}

```

Controlador:

```

public function publish(
    Collection $collection,
    PublishCollectionService $service
): RedirectResponse {
    $this->authorize(
        'update',
        $collection
    );

    $service->publish($collection);

    return back()->with(
        'success',
        'Colección publicada.'
    );
}

```

25. TRANSACCIONES

Utilizar transacciones cuando varias operaciones deben completarse juntas.

```

return DB::transaction(
    function () use ($data) {
        $collection = Collection::create(
            $data
        );

        $collection->entities()->sync(
            $entityIds
        );

        return $collection;
    }
);

```

);

Si alguna operación falla, Laravel revierte todo el bloque.

Debe utilizarse en:

- Clonación.
- Creación con relaciones.
- Guardado de atributos.
- Importaciones.
- Torneos.
- Simulaciones.
- Operaciones financieras futuras.

26. CÓMO AÑADIR UNA RUTA

Archivo:

routes/web.php

26.1. Ruta simple

```
Route::get(
    '/about',
    function () {
        return view('about');
    }
)->name('about');
```

26.2. Ruta con controlador

```
Route::get(
    '/explore',
    [ExploreController::class, 'index']
)->name('community.index');
```

26.3. Ruta POST

```
Route::post(
    '/collections/{collection}/publish',
    [CollectionController::class, 'publish']
)->name('collections.publish');
```

26.4. Resource

```
Route::resource(
    'collections',
    CollectionController::class
);
```

26.5. Orden importante

Las rutas especiales deben declararse antes del resource cuando puedan entrar en conflicto.

Correcto:

```
Route::get(
    'collections/popular',
    [CollectionController::class, 'popular']
)->name('collections.popular');
```

```
Route::resource(
    'collections',
    CollectionController::class
);
```

Laravel recomienda declarar rutas adicionales antes de `Route::resource()` para evitar que una ruta dinámica capture el segmento especial.

26.6. Verificar

```
php artisan route:list
```

```
php artisan route:list --name=collections
```

27. ROUTE MODEL BINDING

Ruta:

```
Route::get(
    '/collections/{collection}',
    [CollectionController::class, 'show']
);
```

Controlador:

```
public function show(
    Collection $collection
): View {
    // Laravel busca automáticamente por ID.
}
```

27.1. Usar slug

En el modelo:

```
public function getRouteKeyName(): string
{
    return 'slug';
}
```

Luego la URL puede ser:

/collections/protagonistas-anime

Advertencia: antes de cambiar esto, debe comprobarse que todos los slugs sean únicos de la forma que requiere la ruta.

28. CÓMO CREAR UNA VISTA

Las vistas Blade se encuentran en:
resources/views/

Ejemplo:

resources/views/universes/index.blade.php

```
<x-app-layout>
  <x-slot name="header">
    Universos
  </x-slot>

  <div class="flex items-center justify-between">
    <div>
      <h2 class="text-2xl font-black text-slate-900">
        Mis universos
      </h2>

      <p class="mt-2 text-slate-500">
        Administra los universos de OmniMerge.
      </p>
    </div>

    <a
      href="{{ route('universes.create') }}"
      class="rounded-xl bg-indigo-600 px-5 py-3 font-bold text-white"
    >
      Nuevo universo
    </a>
  </div>
</x-app-layout>
```

29. PARTIALS Y COMPONENTES

29.1. Partial

Se utiliza cuando un fragmento pertenece a un módulo.

Ejemplo:

resources/views/collections/partials/form.blade.php

Incluir:

```
@include('collections.partials.form')
```

29.2. Componente

Se utiliza cuando se reutiliza en varios módulos.

Ejemplo:

resources/views/components/status-badge.blade.php

Uso:

```
<x-status-badge :status="$collection->status" />
```

29.3. Cuándo usar cada uno

Caso	Usar
Formulario de colecciones	Partial
Formulario de atributos	Partial
Badge de estado	Componente
Mensaje de error	Componente
Modal reutilizable	Componente
Tarjeta exclusiva de comunidad	Partial

30. CÓMO MODIFICAR EL MENÚ

Archivo:

resources/views/partials/sidebar.blade.php

Ejemplo:

```
<a
  href="{{ route('universes.index') }}"
  class="{{
    request()->routeIs('universes.*')
      ? 'bg-indigo-500 text-white'
      : 'text-slate-300 hover:bg-slate-900'
  }}"
  flex items-center gap-3 rounded-xl px-3 py-3"
>
  <span><img alt="Universe icon" data-bbox="200 738 225 755"/></span>
  Universos
</a>
```

Para eliminar una opción, quitar solamente el enlace no elimina el módulo.

El módulo seguirá accesible por URL.

Para deshabilitarlo completamente debe revisarse:

Ruta

Policy

Controlador
Sidebar
Dashboard

31. CÓMO MODIFICAR EL DISEÑO

La interfaz utiliza Tailwind CSS.

Ejemplo:

```
class="rounded-2xl border border-slate-200 bg-white p-6 shadow-sm"
```

31.1. Cambiar color principal

Buscar clases:

indigo-500

indigo-600

indigo-700

Puede cambiarse por:

violet

blue

emerald

rose

Ejemplo:

bg-violet-600

hover:bg-violet-700

text-violet-600

31.2. Evitar cambios globales desordenados

No reemplazar todas las palabras **indigo** sin revisar.

Puede afectar:

- Botones.
- Estados.
- Fondos.
- Formularios.
- Componentes.
- Comunidad.
- Menú.

31.3. Componentes centralizados

Para mantener coherencia es recomendable crear:

components/button-primary.blade.php

components/button-danger.blade.php

components/card.blade.php

components/form-input.blade.php

components/modal.blade.php

32. FORMULARIOS BLADE

32.1. Crear

```
<form
  method="POST"
  action="{{ route('collections.store') }}"
>
  @csrf
</form>
```

32.2. Actualizar

```
<form
  method="POST"
  action="{{ route(
    'collections.update',
    $collection
  ) }}"
>
  @csrf
  @method('PUT')
</form>
```

32.3. Eliminar

```
<form
  method="POST"
  action="{{ route(
    'collections.destroy',
    $collection
  ) }}"
>
  @csrf
  @method('DELETE')
</form>
```

Todo formulario HTML que modifique información debe incluir protección CSRF. Blade proporciona la directiva `@csrf` para generar el token oculto correspondiente.

32.4. Con imágenes

```
<form
  method="POST"
  enctype="multipart/form-data"
>
```

Sin `multipart/form-data`, el archivo no se enviará.

33. MOSTRAR ERRORES DE VALIDACIÓN

Ejemplo por campo:

```
@error('name')
  <p class="mt-2 text-sm text-red-600">
    {{ $message }}
  </p>
@enderror
```

Ejemplo general:

```
@if ($errors->any())
  <div class="rounded-xl bg-red-50 p-4 text-red-700">
    <ul>
      @foreach ($errors->all() as $error)
        <li>{{ $error }}</li>
      @endforeach
    </ul>
  </div>
@endif
```

34. CARGA Y MODIFICACIÓN DE IMÁGENES

34.1. Request

use Illuminate\Validation\Rules\File;

```
'image' => [
    'nullable',

    File::image()
        ->types([
            'jpg',
            'jpeg',
            'png',
            'webp',
        ])
        ->max('4mb'),
],
```

34.2. Guardar

```
if ($request->hasFile('image')) {
    $data['image'] = $request
        ->file('image')
        ->store(
            'collections',
```

```

        'public'
    );
}

```

Laravel permite guardar archivos cargados en un disco configurado mediante los métodos del archivo subido.

34.3. Actualizar

```

if ($request->hasFile('image')) {
    if ($collection->image) {
        Storage::disk('public')
            ->delete($collection->image);
    }

    $data['image'] = $request
        ->file('image')
        ->store(
            'collections',
            'public'
        );
}

```

34.4. Eliminar imagen

```

if ($request->boolean('remove_image')) {
    if ($collection->image) {
        Storage::disk('public')
            ->delete($collection->image);
    }

    $data['image'] = null;
}

```

34.5. Eliminar recurso

```

if ($collection->image) {
    Storage::disk('public')
        ->delete($collection->image);
}

```

```

$collection->delete();

```

34.6. Mostrar

```

@if ($collection->image_url)
    name }}"
    >
@endif

```

>
@endif

35. CARPETAS DE IMÁGENES

storage/app/public/entities/
storage/app/public/attributes/
storage/app/public/attribute-options/
storage/app/public/collections/

El enlace público:
public/storage

Crear:
php artisan storage:link

Si ya existe:
The [public/storage] link already exists

no es necesario volver a crearlo.

36. CÓMO AÑADIR BÚSQUEDA

Controlador:

```
$search = trim(  
    (string) $request->input('search')  
);
```

```
$collections = Collection::query()  
    ->ownedBy($request->user())  
    ->when(  
        $search,  
        fn ($query) => $query->where(  
            function ($subquery) use ($search) {  
                $subquery  
                    ->where(  
                        'name',  
                        'like',  
                        "%{$search}%"  
                    )  
                    ->orWhere(  
                        'description',  
                        'like',  
                        "%{$search}%"  
                    );  
            }  
        )  
    );
```

```

    }
  )
)
->paginate(12)
->withQueryString();

```

Vista:

```

<form method="GET">
  <input
    type="search"
    name="search"
    value="{{ $search }}"
  >

  <button type="submit">
    Buscar
  </button>
</form>

```

37. CÓMO AÑADIR FILTRO

Ejemplo por estado.

Controlador:

```
$status = $request->input('status');
```

```

$query->when(
  $status,
  fn ($query) => $query->where(
    'status',
    $status
  )
);

```

Vista:

```

<select name="status">
  <option value="">Todos</option>

  <option
    value="ACTIVE"
    @selected($status === 'ACTIVE')
  >
    Activos
  </option>

```

```

<option
  value="INACTIVE"
  @selected($status === 'INACTIVE')
>
  Inactivos
</option>
</select>

```

Si se filtra frecuentemente por un campo, conviene añadir un índice.

```

$table->index([
  'user_id',
  'status',
]);

```

38. PAGINACIÓN

Controlador:

```

$collections = $query
->paginate(12)
->withQueryString();

```

Vista:

```

{{ $collections->links() }}

```

`withQueryString()` conserva búsqueda y filtros al cambiar de página.

39. ATRIBUTOS DINÁMICOS: QUÉ MODIFICAR

39.1. Añadir nuevo tipo de dato

Ejemplo:

URL

Modificar:

Migración, si existe una restricción

StoreAttributeRequest

UpdateAttributeRequest

Formulario de atributo

EntityAttributeValueService

Formulario de valores de entidad

EntityAttributeValue::displayValue()

Tests

39.2. Request

```

Rule::in([
  'TEXT',

```



```
'LONG_TEXT',
'INTEGER',
'DECIMAL',
'BOOLEAN',
'DATE',
'COLOR',
'OPTION',
'URL',
]),
```

39.3. Formulario

```
<option value="URL">
  Dirección web
</option>
```

39.4. Guardado

Podría reutilizar:
text_value

```
case 'URL':
  $value->text_value = $rawValue;
  break;
```

39.5. Validación

```
case 'URL':
  validator(
    ['value' => $rawValue],
    ['value' => ['nullable', 'url']]
  )->validate();
  break;
```

39.6. Vista

```
@if ($attribute->data_type === 'URL')
  <input
    type="url"
    name="attributes[{{ $attribute->id }}]"
  >
@endif
```

40. MULTISELECCIÓN

Cuando:
allows_multiple = true

el formulario debe enviar:
attributes[ID_ATRIBUTO][]

Ejemplo:

```
<input
  type="checkbox"
  name="attributes[{{ $attribute->id }}]"
  value="{{ $option->id }}"
>
```

Cuando es único:

```
<input
  type="radio"
  name="attributes[{{ $attribute->id }}]"
  value="{{ $option->id }}"
>
```

El Service debe eliminar los valores anteriores y guardar todas las opciones seleccionadas.

41. OPCIONES DE ATRIBUTOS

Cuando se quiera cambiar una opción, revisar:

```
app/Models/AttributeOption.php
app/Http/Controllers/Attributes/AttributeOptionController.php
app/Http/Requests/Attributes/
resources/views/attribute-options/
database/migrations/
```

41.1. Añadir nuevo campo

Ejemplo:

official_url

Crear migración:

```
php artisan make:migration add_official_url_to_attribute_options_table
--table=attribute_options
```

Luego modificar:

\$fillable

Request

Formulario

Show

CloneService

Tests

Es importante modificar también **CommunityCloneService**, porque si no se añade el nuevo campo, las opciones clonadas perderán esa información.

42. COLECCIONES: QUÉ MODIFICAR

Añadir una característica visual

Modificar:

collections migration

Collection model

StoreCollectionRequest

UpdateCollectionRequest

CollectionController

collections/partials/form.blade.php

collections/index.blade.php

collections/show.blade.php

community/collection.blade.php

CommunityCloneService

tests

Añadir una nueva relación

Ejemplo:

Collection hasMany Section

Crear:

sections

collection_id

Modificar:

Collection model

Section model

Controller

Views

CloneService

Tests

43. COMUNIDAD: SEGURIDAD Y CAMBIOS

El explorador debe mostrar únicamente contenido autorizado.

Entidades:

->where('visibility', 'PUBLIC')

->where('status', 'ACTIVE')

->whereNotNull('published_at')

Colecciones:

->where('visibility', 'PUBLIC')

```
->where('status', 'ACTIVE')
->whereNotNull('published_at')
```

Atributos:

```
->where('scope', 'PUBLIC')
->where('status', 'ACTIVE')
->whereNotNull('published_at')
```

Nunca usar en comunidad:

```
Entity::all();
Collection::all();
Attribute::all();
```

43.1. Añadir favoritos

Crear migración o reutilizar:
community_interactions

Tipo:

FAVORITE

Crear rutas:

```
Route::post(
    '/explore/entities/{entity}/favorite',
    [ExploreController::class, 'favoriteEntity']
)->name('community.entities.favorite');
```

Controlador:

```
DB::table('community_interactions')
    ->updateOrCreate(
        [
            'user_id' => $request->user()->id,
            'content_type' => 'ENTITY',
            'content_id' => $entity->id,
            'interaction_type' => 'FAVORITE',
        ],
        [
            'updated_at' => now(),
            'created_at' => now(),
        ]
    );
```

44. CLONACIÓN: QUÉ NO OLVIDAR

Cuando se añade un nuevo campo a:

Entity
Attribute
AttributeOption
Collection

debe revisarse:
CommunityCloneService.php

Ejemplo: se añade **subtitle** a entidades.

Si no se modifica el Service:

```
$entity = $user->entities()->create([  
    'name' => $source->name,  
]);
```

la copia no incluirá **subtitle**.

Debe añadirse:

```
'subtitle' => $source->subtitle,
```

44.1. Operaciones que deben clonarse

Entidad:

Datos
Imagen
Tipo
Atributos
Opciones
Valores
Metadatos

Atributo:

Configuración
Imagen
Opciones
Jerarquía
Reglas

Colección:

Datos
Portada
Entidades públicas
Orden

44.2. Operaciones que no deben clonarse automáticamente

ID

user_id original

views_count
clones_count
published_at
visibilidad pública
comentarios
reportes
interacciones

La copia debe quedar privada inicialmente.

45. BORRAR UN REGISTRO

45.1. Soft delete

```
$collection->delete();
```

No elimina físicamente el registro si el modelo utiliza **SoftDeletes**.

45.2. Restaurar

```
$collection = Collection::withTrashed()  
->findOrFail($id);
```

```
$collection->restore();
```

45.3. Eliminar definitivamente

```
$collection->forceDelete();
```

Debe utilizarse con extremo cuidado.

45.4. Archivos

Soft delete no elimina automáticamente las imágenes.

Decidir:

- Mantener archivo para restauración.
- Eliminar archivo inmediatamente.
- Eliminar archivo al ejecutar **forceDelete**.

Para permitir restauración, conviene no eliminar la imagen en el soft delete.

46. ELIMINAR UN MÓDULO COMPLETO

Ejemplo: eliminar **AttributeGroup**.

No basta con borrar la vista.

Revisar:

routes/web.php

sidebar.blade.php

AttributeGroupController

Requests

Policy

Model

Views

Relationships
Migrations
Factories
Seeders
Tests
Dashboard
CommunityCloneService

Crear una migración nueva para eliminar tablas.
php artisan make:migration drop_attribute_group_tables

```
public function up(): void
{
    Schema::dropIfExists(
        'attribute_group_attribute'
    );

    Schema::dropIfExists(
        'attribute_groups'
    );
}
```

No borrar una migración antigua ya compartida si forma parte del historial del proyecto.

47. TRANSFERIR EL PROYECTO A OTRA COMPUTADORA

47.1. Código

Utilizar GitHub:
git clone <https://github.com/grverlearner/OmniMerge.git>

47.2. Base de datos

Opciones:

Reconstruir vacía

php artisan migrate

Transferir datos

Exportar MySQL:

```
mysqldump -u root -p omnimerge > omnimerge.sql
```

Importar:

```
mysql -u root -p omnimerge < omnimerge.sql
```

47.3. Imágenes

Copiar:

storage/app/public/

No basta con copiar la base de datos. La base solo guarda las rutas de los archivos.

47.4. Entorno

Crear un nuevo `.env`.

No copiar un `.env` con credenciales sensibles sin protección.

47.5. Enlace de storage

En la nueva computadora:

`php artisan storage:link`

48. EXPORTAR SOLO UNA FUNCIONALIDAD

Ejemplo: transferir el módulo `Collections` a otro proyecto.

Copiar:

Collection model

CollectionController

Requests

Policy

Views

Migrations

Routes

Factories

Tests

Service relacionado

También revisar dependencias:

User

Entity

collection_entity

Storage

Components Blade

Layout

Sidebar

No copiar únicamente `CollectionController.php`, porque dependerá de clases que no existen en el otro proyecto.

49. ACTUALIZAR LARAVEL O DEPENDENCIAS

49.1. Antes de actualizar

`git status`

`git add .`

`git commit -m "chore: guardar estado antes de actualizar"`

Crear rama:

`git checkout -b chore/update-dependencies`

49.2. Composer

Ver dependencias:
composer outdated

Actualizar:
composer update

49.3. NPM

Ver:
npm.cmd outdated

Actualizar:
npm.cmd update

49.4. Después

php artisan optimize:clear
php artisan migrate:status
php artisan route:list
php artisan test
npm.cmd run build

No actualizar versiones principales sin revisar la guía oficial de actualización.

50. ARCHIVOS QUE NO DEBEN EDITARSE DIRECTAMENTE

vendor/

Contiene paquetes de Composer.
Los cambios se pierden con:
composer install
composer update

node_modules/

Contiene paquetes de NPM.
Los cambios se pierden con:
npm.cmd install

public/build/

Generado por Vite.
Modificar:
resources/css/
resources/js/
resources/views/

Luego:

npm.cmd run build

bootstrap/cache/

Contiene caché generado.

storage/framework/

Contiene sesiones, caché y vistas compiladas.

51. ARCHIVOS QUE DEBEN MODIFICARSE CON CUIDADO

routes/web.php

Un error puede dejar inaccesible toda la aplicación.

app/Models/User.php

Es utilizado por todos los módulos.

Migración inicial de usuarios

No modificar después de compartirla sin conocer sus consecuencias.

config/filesystems.php

Afecta imágenes y archivos.

app/Services/Community/CommunityCloneService.php

Una modificación incorrecta puede generar copias incompletas.

EntityAttributeValueService.php

Afecta todos los atributos de las entidades.

resources/views/layouts/app.blade.php

Afecta todas las páginas autenticadas.

sidebar.blade.php

Un error Blade puede romper el panel completo.

52. PRUEBAS

Laravel incluye soporte para pruebas Unit y Feature. Las pruebas Feature permiten verificar flujos completos con rutas, autenticación, base de datos y respuestas HTTP, mientras que las Unit se centran en piezas aisladas. Pueden ejecutarse mediante **php artisan test**.

52.1. Crear prueba

```
php artisan make:test CollectionTest
```

52.2. Ejemplo

```
<?php
```

```
namespace Tests\Feature;
```

```
use App\Models\Collection;
```

```
use App\Models\User;
```

```
use Illuminate\Foundation\Testing\RefreshDatabase;
```

```
use Tests\TestCase;
```

```
class CollectionTest extends TestCase
```

```
{
```

```

use RefreshDatabase;

public function test_user_can_create_collection(): void
{
    $user = User::factory()->create();

    $response = $this
        ->actingAs($user)
        ->post(
            route('collections.store'),
            [
                'name' => 'Favoritos',
                'code' => 'FAVORITOS',
                'slug' => 'favoritos',
                'visibility' => 'PRIVATE',
                'status' => 'ACTIVE',
                'sort_order' => 0,
            ]
        );

    $response->assertRedirect();

    $this->assertDatabaseHas(
        'collections',
        [
            'user_id' => $user->id,
            'name' => 'Favoritos',
        ]
    );
}
}

```

52.3. Probar seguridad

```

public function test_user_cannot_edit_foreign_collection(): void
{
    $owner = User::factory()->create();
    $intruder = User::factory()->create();

    $collection = Collection::factory()
        ->for($owner)
        ->create();

    $response = $this
        ->actingAs($intruder)

```

```

        ->get(
            route(
                'collections.edit',
                $collection
            )
        );

$response->assertForbidden();
}

```

52.4. Probar imágenes

```

use Illuminate\Http\UploadedFile;
use Illuminate\Support\Facades\Storage;

```

```

Storage::fake('public');

```

```

$image = UploadedFile::fake()
    ->image('portada.jpg');

```

```

$response = $this
    ->actingAs($user)
    ->post(
        route('collections.store'),
        [
            'name' => 'Anime',
            'code' => 'ANIME',
            'slug' => 'anime',
            'image' => $image,
            'visibility' => 'PRIVATE',
            'status' => 'ACTIVE',
        ]
    );

```

```

$collection = Collection::first();

```

```

Storage::disk('public')
    ->assertExists($collection->image);

```

52.5. Ejecutar

```

php artisan test

```

Solo Feature:

```

php artisan test --testsuite=Feature

```

Detener en primer error:
php artisan test --stop-on-failure

53. FACTORIES

Cuando se añade un campo obligatorio, actualizar la factory.

Ejemplo:

database/factories/UserFactory.php

```
return [  
    'name' => fake()->name(),  
    'username' => fake()  
        ->unique()  
        ->userName(),  
    'email' => fake()  
        ->unique()  
        ->safeEmail(),  
    'password' => static::$password  
        ??= Hash::make('password'),  
    'role' => 'USER',  
    'status' => 'ACTIVE',  
];
```

Si un campo **NOT NULL** no está en la factory, los tests pueden fallar.

54. SEEDERS

Crear:

php artisan make:seeder DemoAttributeSeeder

Ejemplo:

```
public function run(): void  
{  
    $user = User::first();  
  
    $attribute = $user  
        ->attributes()  
        ->create([  
            'name' => 'Anime',  
            'code' => 'ANIME',  
            'slug' => 'anime',  
            'data_type' => 'OPTION',  
            'value_source' => 'CATALOG',  
            'display_style' => 'MULTISELECT',  
            'allows_multiple' => true,  
        ])
```

```

        'scope' => 'PRIVATE',
        'status' => 'ACTIVE',
    ]);

    $attribute->options()->createMany([
        [
            'name' => 'Naruto',
            'code' => 'NARUTO',
            'status' => 'ACTIVE',
        ],
        [
            'name' => 'One Piece',
            'code' => 'ONE_PIECE',
            'status' => 'ACTIVE',
        ],
    ]);
}

```

Ejecutar:

```
php artisan db:seed --class=DemoAttributeSeeder
```

Reconstruir con seed:

```
php artisan migrate:fresh --seed
```

55. COPIAS DE SEGURIDAD

Antes de cambios grandes:

Código

```
git add .
```

```
git commit -m "chore: respaldo antes de cambio estructural"
```

```
git push origin main
```

Base de datos

```
mysqldump -u root -p omnimerge > backup_omnimerge.sql
```

Imágenes

Copiar:

```
storage/app/public/
```

Entorno

Guardar `.env` de forma privada.

56. GIT: FLUJO RECOMENDADO

Actualizar:

git checkout main
git pull origin main

Crear rama:
git checkout -b feat/universes

Revisar:
git status
git diff

Añadir:
git add .

Commit:
git commit -m "feat(universes): implementar modulo de universos"

Publicar:
git push origin feat/universes

Commits recomendados

feat: nueva funcionalidad
fix: corrección
docs: documentación
refactor: reorganización sin cambiar comportamiento
test: pruebas
chore: configuración o mantenimiento
style: formato visual
security: mejora de seguridad

57. GUÍA DE ERRORES FRECUENTES

View [...] not found

Causa:
La vista no existe o está en una carpeta incorrecta.

Comprobar:
Get-ChildItem resources\views -Recurse

Limpiar:
php artisan view:clear
php artisan optimize:clear

Route [...] not defined

Comprobar:

php artisan route:list

Revisar:

routes/web.php

nombre de la ruta

import del controlador

grupo auth

Undefined method authorize

Revisar:

app/Http/Controllers/Controller.php

Debe usar:

use AuthorizesRequests;

Undefined method url

Tipar:

```
/** @var \Illuminate\Filesystem\FilesystemAdapter $disk */  
$disk = Storage::disk('public');
```

Undefined method user

Comprobar import:

use Illuminate\Http\Request;

No utilizar:

use Symfony\Component\HttpFoundation\Request;

Tipar:

```
/** @var User $user */  
$user = $request->user();
```

MassAssignmentException

Añadir el campo a:

protected \$fillable = [];

Unknown column

Ejecutar:

php artisan migrate:status

php artisan migrate

Table already exists

Revisar migraciones duplicadas.

En desarrollo:

php artisan migrate:fresh

Imagen no aparece

Comprobar:

php artisan storage:link

Comprobar:

enctype="multipart/form-data"

storage/app/public/

public/storage/

Cambios Blade no aparecen

php artisan view:clear

php artisan optimize:clear

npm.cmd run dev

Cambios CSS no aparecen

npm.cmd run build

Error 403

Revisar:

Policy

authorize()

user_id

estado del usuario

Error 404 en recurso existente

Revisar:

Route Model Binding

SoftDeletes

scope público

slug

ID

58. SEGURIDAD MÍNIMA OBLIGATORIA

Toda nueva funcionalidad debe cumplir:

Autenticación

Policy

Validación

Propiedad de registros

CSRF

Consultas filtradas

Archivos validados

Mass assignment controlado

Salida escapada

Pruebas

58.1. No confiar en IDs del navegador

Incorrecto:

```
$entity = Entity::findOrFail(
    $request->entity_id
);
```

Mejor:

```
$entity = Entity::query()
    ->where(
        'user_id',
        $request->user()->id
    )
    ->findOrFail(
        $request->entity_id
    );
```

58.2. No usar datos sin validar

Incorrecto:

```
Collection::create(
    $request->all()
);
```

Correcto:

```
Collection::create(
    $request->validated()
);
```

58.3. No mostrar contenido privado

Incorrecto:

```
Entity::findOrFail($id);
```

En comunidad:

```
Entity::query()
    ->where('visibility', 'PUBLIC')
    ->where('status', 'ACTIVE')
    ->whereNotNull('published_at')
    ->findOrFail($id);
```

58.4. Escapar HTML

Blade escapa:

```
{{ $entity->description }}
```

No usar sin necesidad:

{!! \$entity->description !!}

La segunda opción puede permitir HTML peligroso.

59. CHECKLIST PARA AÑADIR UNA FUNCIONALIDAD

Antes:

- ☐ Definir qué problema resuelve
- ☐ Identificar tablas afectadas
- ☐ Identificar relaciones
- ☐ Definir permisos
- ☐ Definir validaciones
- ☐ Crear rama Git
- ☐ Crear respaldo

Durante:

- ☐ Migración
- ☐ Modelo
- ☐ Relaciones
- ☐ Request
- ☐ Policy
- ☐ Controller
- ☐ Service
- ☐ Routes
- ☐ Views
- ☐ Sidebar
- ☐ CloneService, si aplica
- ☐ Factory
- ☐ Seeder
- ☐ Tests

Después:

- ☐ php artisan migrate
- ☐ php artisan route:list
- ☐ php artisan optimize:clear
- ☐ npm.cmd run build
- ☐ php artisan test
- ☐ Prueba manual
- ☐ git status
- ☐ git diff
- ☐ commit
- ☐ push

60. CHECKLIST PARA MODIFICAR UN CAMPO

- ☐ Crear nueva migración
- ☐ Actualizar fillable
- ☐ Actualizar casts
- ☐ Actualizar Request Store
- ☐ Actualizar Request Update
- ☐ Actualizar create
- ☐ Actualizar edit
- ☐ Actualizar show
- ☐ Actualizar index
- ☐ Actualizar Controller
- ☐ Actualizar Service
- ☐ Actualizar CloneService
- ☐ Actualizar Factory
- ☐ Actualizar Seeder
- ☐ Actualizar tests

61. CHECKLIST PARA BORRAR ALGO

- ☐ Buscar todas las referencias
- ☐ Comprobar relaciones
- ☐ Comprobar claves foráneas
- ☐ Crear backup
- ☐ Crear migración nueva
- ☐ Eliminar de formularios
- ☐ Eliminar de Requests
- ☐ Eliminar de modelos
- ☐ Eliminar de Services
- ☐ Eliminar de vistas
- ☐ Eliminar de tests
- ☐ Probar rollback

62. CHECKLIST PARA PUBLICAR CONTENIDO

- ☐ visibility o scope = PUBLIC
- ☐ status = ACTIVE
- ☐ published_at no nulo
- ☐ allow_cloning correctamente definido
- ☐ Policy revisada
- ☐ Consulta pública filtrada
- ☐ Datos privados ocultos
- ☐ Imagen accesible
- ☐ Prueba con segundo usuario

63. CHECKLIST PARA TRANSFERIR

- ☐ Código subido a GitHub
- ☐ Base de datos exportada, si se necesitan datos
- ☐ storage/app/public copiado
- ☐ .env.example actualizado
- ☐ .env transferido de forma privada
- ☐ composer install
- ☐ npm install
- ☐ php artisan key:generate
- ☐ php artisan migrate
- ☐ php artisan storage:link
- ☐ npm run build
- ☐ php artisan test

64. MAPA DE IMPACTO DE OMNIMERGE

Si se modifica **Entity**

Revisar:

EntityController

StoreEntityRequest

UpdateEntityRequest

EntityPolicy

Entity views

Collection views

Community views

CommunityCloneService

EntityAttributeValueService

Tests

Si se modifica **Attribute**

Revisar:

AttributeController

Attribute Requests

Attribute views

Entity attributes form

AttributeOption

AttributeGroup

Community

CommunityCloneService

Tests

Si se modifica **AttributeOption**

Revisar:

AttributeOptionController

Requests

Options views

Entity attributes form

EntityAttributeValueService

Community attribute detail

CloneService

Tests

Si se modifica Collection

Revisar:

CollectionController

Requests

Policy

Views

Community views

CloneService

Entity relationships

Tests

Si se modifica User

Revisar:

Auth

Factories

Tests

Policies

Dashboard

Community creator data

Profile

Relations

65. MEJORAS RECOMENDADAS PARA LA ARQUITECTURA

65.1. Enums PHP

Actualmente se utilizan strings:

ACTIVE

PRIVATE

PUBLIC

OPTION

Puede mejorarse creando:

app/Enums/EntityStatus.php

app/Enums/Visibility.php

app/Enums/AttributeDataType.php

Ejemplo:

```
namespace App\Enums;
```

```
enum Visibility: string
{
    case PRIVATE = 'PRIVATE';
    case PUBLIC = 'PUBLIC';
    case UNLISTED = 'UNLISTED';
}
```

Modelo:

```
'visibility' => Visibility::class,
```

65.2. Actions

Para operaciones medianas:

app/Actions/Collections/CreateCollection.php

app/Actions/Entities/PublishEntity.php

65.3. Observers

Para eliminar imágenes o actualizar fechas automáticamente:

```
php artisan make:observer CollectionObserver --model=Collection
```

65.4. Eventos

EntityPublished

CollectionCloned

TournamentCompleted

65.5. Jobs

Para tareas pesadas:

CloneLargeCollection

GenerateTournament

ExportUniverse

65.6. Notificaciones

Contenido clonado

Comentario recibido

Torneo finalizado

66. PRÓXIMOS MÓDULOS

Relaciones entre atributos

attribute_relationships

attribute_conditions
attribute_option_relationships

Universos

universes
universe_entities

Temporadas

seasons
season_entities

Torneos

tournaments
tournament_participants
tournament_rounds
tournament_matches
match_results

Simulación

simulation_rules
simulation_events
simulation_results

Comunidad avanzada

favorites
comments
reports
follows
notifications

67. PRINCIPIO FINAL DE MANTENIMIENTO

Antes de modificar OmniMerge, responder estas preguntas:

¿Este cambio modifica la base de datos?

¿Este cambio necesita validación?

¿Quién puede ejecutar la acción?

¿Afecta a contenido público?

¿Afecta a clonación?

¿Afecta a imágenes?

¿Afecta a relaciones?

¿Afecta a formularios?

¿Afecta a pruebas?

¿Puede perderse información?

Si el cambio modifica datos persistentes, nunca debe realizarse únicamente en una vista.
Si el cambio modifica permisos, nunca debe resolverse únicamente ocultando un botón.
Si el cambio modifica contenido público, debe comprobarse con otro usuario.
Si el cambio modifica imágenes, debe revisarse el almacenamiento y la eliminación de archivos.
Si el cambio modifica atributos, debe revisarse el sistema de valores de entidades.
Si el cambio modifica entidades, atributos, opciones o colecciones, debe revisarse el servicio de clonación.

68. COMANDOS DE REFERENCIA RÁPIDA

Ejecutar

```
php artisan serve  
npm.cmd run dev
```

Instalar

```
composer install  
npm.cmd install
```

Compilar

```
npm.cmd run build
```

Migraciones

```
php artisan migrate  
php artisan migrate:status  
php artisan migrate:rollback  
php artisan migrate:fresh  
php artisan migrate:fresh --seed
```

Caché

```
php artisan optimize:clear  
php artisan view:clear  
php artisan route:clear  
php artisan config:clear
```

Rutas

```
php artisan route:list
```

Storage

```
php artisan storage:link
```

Tests

```
php artisan test  
php artisan test --stop-on-failure
```

Modelos

php artisan make:model Nombre -mf

Controladores

php artisan make:controller Carpeta/NombreController --resource --model=Nombre

Requests

php artisan make:request Carpeta/StoreNombreRequest

php artisan make:request Carpeta/UpdateNombreRequest

Policies

php artisan make:policy NombrePolicy --model=Nombre

Tests

php artisan make:test NombreTest

Seeders

php artisan make:seeder NombreSeeder

Estado Git

git status

git diff

git log --oneline

Guardar cambios

git add .

git commit -m "feat: descripción"

git push origin main

69. CONCLUSIÓN

OmniMerge utiliza una arquitectura modular donde cada funcionalidad se distribuye entre la base de datos, modelos, validaciones, permisos, controladores, servicios, rutas, vistas y pruebas.

El proyecto puede ampliarse de manera segura siempre que se respete esta separación de responsabilidades.

Los principios fundamentales son:

No modificar vendor.

No publicar .env.

No editar directamente la base de datos sin migraciones.

No confiar en IDs enviados por el navegador.

No guardar datos sin validar.

No depender solo de botones ocultos para la seguridad.

No olvidar actualizar CloneService.

No eliminar datos sin respaldo.

No desplegar sin pruebas.

Siguiendo este manual será posible:

- Reconstruir OmniMerge.
- Entender su arquitectura.
- Añadir módulos.
- Cambiar campos.
- Modificar vistas.
- Actualizar la seguridad.
- Transferir el proyecto.
- Corregir errores.
- Mantener la base de datos.
- Preparar futuras versiones.

La idea principal es que cada cambio se realice en la capa correcta y que todos los componentes relacionados sean revisados antes de considerar que la funcionalidad está completa.